

INTRODUCCIÓN A CURSORES EN SQL SERVER

Los cursores son una herramienta de SQL que nos permite recorrer el resultado de una consulta SQL y realizar operaciones en cada paso de ésta.

- En algunos SGDB es posible la abertura de cursores de datos desde el propio entorno de trabajo, para ello se utilizan, normalmente procedimientos almacenados. La sintaxis para definir un cursor es la siguiente:

```
DECLARE nombre-cursor FOR  
especificacion-consulta  
[ORDER BY]
```

Por ejemplo:

```
DECLARE Mi_Cursor FOR  
SELECT num_emp, nombre, puesto, salario  
FROM empleados  
WHERE num_dept = 'informatica'
```

(Este comando es meramente declarativo, simplemente especifica las filas y columnas que se van a recuperar. La consulta se ejecuta cuando se abre o se activa el cursor. La cláusula [ORDER BY] es opcional y especifica una ordenación para las filas del cursor; si no se especifica, la ordenación de las filas es definida el gestor de SGDB.)

- Para abrir o activar un cursor se utiliza el comando OPEN del SQL, la sintaxis en la siguiente:

```
OPEN nombre-cursor [USING lista-variables]
```

(Al abrir el cursor se evalúa la consulta que aparece en su definición, utilizando los valores actuales de cualquier parámetro referenciado en la consulta, para producir una colección de filas. El puntero se posiciona delante de la primera fila de datos (registro actual), esta sentencia no recupera ninguna fila.)

- Una vez abierto el cursos se utiliza la cláusula FETCH para recuperar las filas del cursor, la sintaxis es la siguiente:

```
FETCH nombre-cursor INTO lista-variables
```

(Lista - variables son las variables que van a contener los datos recuperados de la fila del cursor, en la definición deben ir separadas por comas. En la lista de variables se deben definir tantas variables como columnas tenga la fila a recuperar.)

- Para cerrar un cursor se utiliza el comando CLOSE, este comando hace desaparecer el puntero sobre el registro actual. La sintaxis es:

CLOSE nombre-cursor

- Por último, y para eliminar el cursor se utiliza el comando DROP CURSOR. Su sintaxis es la siguiente:

DROP CURSOR nombre-cursor

Ejemplos (sobre SQL-SERVER):

```
'Abrir un cursor y recorrela
DECLARE Employee_Cursor CURSOR FOR
SELECT LastName, FirstName
FROM Northwind.dbo.Employees
WHERE LastName like 'B%'
```

```
OPEN Employee_Cursor
```

```
FETCH NEXT FROM Employee_Cursor
```

```
WHILE @@FETCH_STATUS = 0
BEGIN
    FETCH NEXT FROM Employee_Cursor
END
CLOSE Employee_Cursor
DEALLOCATE Employee_Cursor
```

```
'Abrir un cursor e imprimir su contenido
SET NOCOUNT ON
DECLARE
@au_id varchar(11),
@au_fname varchar(20),
@au_lname varchar(40),
@message varchar(80),
@title varchar(80)
```

```
PRINT "----- Utah Authors report -----"
```

```
DECLARE authors_cursor CURSOR FOR
SELECT au_id, au_fname, au_lname
FROM authors
WHERE state = "UT"
```

```
ORDER BY au_id
```

```
OPEN authors_cursor  
FETCH NEXT FROM authors_cursor  
INTO @au_id, @au_fname, @au_lname
```

```
WHILE @@FETCH_STATUS = 0  
BEGIN  
PRINT " "  
SELECT  
@message = "----- Books by Author: " +  
@au_fname + " " + @au_lname  
PRINT @message
```

```
DECLARE titles_cursor CURSOR FOR  
SELECT t.title  
FROM titleauthor ta, titles t  
WHERE ta.title_id = t.title_id AND ta.au_id = au_id
```

```
OPEN titles_cursor  
FETCH NEXT FROM titles_cursor INTO @title  
IF @@FETCH_STATUS <> 0  
PRINT " <<No Books>>"  
WHILE @@FETCH_STATUS = 0  
BEGIN  
SELECT @message = " " + @title  
PRINT @message  
FETCH NEXT FROM titles_cursor INTO @title  
END  
CLOSE titles_cursor  
DEALLOCATE titles_cursor
```

```
FETCH NEXT FROM authors_cursor  
INTO @au_id, @au_fname, @au_lname  
END  
CLOSE authors_cursor  
DEALLOCATE authors_cursor  
GO
```

```
'Recorrer un cursor  
USE pubs  
GO  
DECLARE authors_cursor CURSOR FOR  
SELECT au_lname  
FROM authors  
WHERE au_lname LIKE "B%"  
ORDER BY au_lname
```

```
OPEN authors_cursor  
FETCH NEXT FROM authors_cursor
```

```
WHILE @@FETCH_STATUS = 0
BEGIN
FETCH NEXT FROM authors_cursor
END
CLOSE authors_cursor
DEALLOCATE authors_cursor
```

'Recorrer un cursor guardando los valores en variables

USE pubs

GO

DECLARE @au_lname varchar(40)

DECLARE @au_fname varchar(20)

DECLARE authors_cursor CURSOR FOR

SELECT au_lname, au_fname

FROM authors

WHERE au_lname LIKE "B%"

ORDER BY au_lname, au_fname

OPEN authors_cursor

FETCH NEXT FROM authors_cursor INTO @au_lname, @au_fname

WHILE @@FETCH_STATUS = 0

BEGIN

PRINT "Author: " + @au_fname + " " + @au_lname

FETCH NEXT FROM authors_cursor

INTO @au_lname, @au_fname

END

CLOSE authors_cursor

DEALLOCATE authors_cursor

Otro Ejemplo

*/*Este cursor deja las contraseñas iguales al nombre de usuario.*

CREATE PROCEDURE EJEMPLO

AS

BEGIN

*/*La tabla Cliente tiene estos tres campos: CliCod, CliUser, CliPass */*

-- declaramos las variables

declare @cod as int

declare @user as varchar(50)

declare @pass as varchar(50)

-- declaramos un cursor llamado "CURSORITO".

```
-- El select debe contener sólo los campos a utilizar.
declare CURSORITO cursor for
select CliCod, CliUser, CliPass from Cliente

open CURSORITO
-- Avanzamos un registro y cargamos en las variables los valores
-- encontrados en el primer registro

fetch next from CURSORITO into @cod, @user, @pass

while @@fetch_status = 0
begin
    update Cliente set CliPass= @user where CliCod=@cod
    -- Avanzamos otro registro
    fetch next from CURSORITO into @cod, @rut, @nombres
end

-- cerramos el cursor
close CURSORITO

END
```